



SETUP GUIDE

Cost Analysis and Benchmarking Tools for ML Deployment

SECTION 1: OVERVIEW

What we build:

- Python A Python-based ML cost analysis toolkit with 3 scripts
- `cost_calculator.py` — compares deployment costs across hardware options
- `benchmark.py` — measures actual model inference latency and throughput
- `tco_analysis.py` — generates Total Cost of Ownership breakdown
- A sample scikit-learn model for benchmarking demonstrations

Purpose:

- Transform gut-feel deployment decisions into data-driven recommendations
- Produce stakeholder-ready comparison tables
- Shows how changing one parameter (traffic, hardware, team size)

changes the optimal architecture recommendation

SECTION 2: PREREQUISITES

Required:

- Python 3.9+ installed
- pip (Python package manager)
- Text editor (VS Code recommended)
- Terminal/command line access

No cloud account needed — everything runs locally.

No GPU needed — benchmarks run on CPU (demonstrates the measurement approach).

SECTION 3: PROJECT STRUCTURE

```
ml-cost-toolkit/  
├─ src/
```

SECTION 4: FILE CONTENTS



FILE: requirements.txt

```
scikit-learn==1.4.0
pandas==2.2.0
numpy==1.26.4
tabulate==0.9.0
```

FILE: src/__init__.py

FILE: models/train_sample_model.py

```
"""Train a sample model for benchmarking demonstrations."""

import pickle
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification

def create_sample_model(model_path: str = "models/sample_model.pkl"):
    """Train and save a sample Random Forest model."""
    # Create synthetic dataset (1000 samples, 20 features)
    X, y = make_classification(
        n_samples=1000,
        n_features=20,
        n_informative=15,
        n_classes=3,
        random_state=42,
    )

    # Train model
    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X, y)

    # Save
    with open(model_path, "wb") as f:
        pickle.dump(model, f)

    print(f"Model saved to {model_path}")
    print(f" Features: 20")
    print(f" Classes: 3")
    print(f" Estimators: 100")
    print(f" Model size: {round(len(pickle.dumps(model)) / 1024, 1)} KB")
```

```
return model
```

```
if __name__ == "__main__":  
    create_sample_model()
```

FILE: src/benchmark.py

```
"""Inference benchmarking tool — measures latency and throughput."""
```

```
import time  
import pickle  
import numpy as np  
from tabulate import tabulate
```

```
def load_model(model_path: str = "models/sample_model.pkl"):  
    """Load a trained model."""  
    with open(model_path, "rb") as f:  
        return pickle.load(f)
```

```
def benchmark_inference(model, n_features: int = 20, n_warmup: int = 100,  
                        n_iterations: int = 1000):
```

```
    """Benchmark single-sample inference latency."""  
    # Generate test input  
    test_input = np.random.rand(1, n_features)
```

```
    # Warmup (eliminate cold-start effects)  
    for _ in range(n_warmup):  
        model.predict(test_input)
```

```
    # Timed runs  
    latencies = []  
    for _ in range(n_iterations):  
        start = time.perf_counter()  
        model.predict(test_input)  
        end = time.perf_counter()  
        latencies.append((end - start) * 1000) # Convert to ms
```

```
    return np.array(latencies)
```

```
def benchmark_throughput(model, n_features: int = 20, batch_sizes: list = None):  
    """Benchmark batch inference throughput."""
```

```
if batch_sizes is None:
    batch_sizes = [1, 8, 16, 32, 64, 128]

results = []
for batch_size in batch_sizes:
    test_input = np.random.rand(batch_size, n_features)

    # Warmup
    for _ in range(50):
        model.predict(test_input)

    # Timed
    start = time.perf_counter()
    n_batches = 100
    for _ in range(n_batches):
        model.predict(test_input)
    elapsed = time.perf_counter() - start

    total_inferences = batch_size * n_batches
    throughput = total_inferences / elapsed
    latency_per_batch = (elapsed / n_batches) * 1000 # ms

    results.append({
        "batch_size": batch_size,
        "throughput_per_sec": round(throughput, 1),
        "batch_latency_ms": round(latency_per_batch, 2),
        "per_sample_ms": round(latency_per_batch / batch_size, 3),
    })

return results

def run_benchmark():
    """Run complete benchmark suite and display results."""
    print("=" * 70)
    print("INFERENCE BENCHMARK REPORT")
    print("=" * 70)

    # Load model
    model = load_model()
    print(f"\nModel: Random Forest (100 estimators, 20 features, 3 classes)")
    print(f"Hardware: CPU (local machine)")
    print(f"Benchmark: 1000 iterations after 100 warmup runs\n")
```

```
# Single-sample latency
latencies = benchmark_inference(model)
print("-" * 70)
print("SINGLE-SAMPLE LATENCY (milliseconds)")
print("-" * 70)

stats = [
    ["P50 (median)", f"{np.percentile(latencies, 50):.3f} ms"],
    ["P90", f"{np.percentile(latencies, 90):.3f} ms"],
    ["P95", f"{np.percentile(latencies, 95):.3f} ms"],
    ["P99", f"{np.percentile(latencies, 99):.3f} ms"],
    ["Mean", f"{np.mean(latencies):.3f} ms"],
    ["Std Dev", f"{np.std(latencies):.3f} ms"],
    ["Min", f"{np.min(latencies):.3f} ms"],
    ["Max", f"{np.max(latencies):.3f} ms"],
]
print(tabulate(stats, headers=["Metric", "Value"], tablefmt="grid"))

# Throughput by batch size
print(f"\n{'-' * 70}")
print("THROUGHPUT BY BATCH SIZE")
print("-" * 70)

throughput_results = benchmark_throughput(model)
table_data = [
    [r["batch_size"], r["throughput_per_sec"],
     r["batch_latency_ms"], r["per_sample_ms"]]
    for r in throughput_results
]
print(tabulate(
    table_data,
    headers=["Batch Size", "Throughput/sec", "Batch Latency (ms)", "Per Sample (ms)"],
    tablefmt="grid",
))

print(f"\n{'=' * 70}")
print("BENCHMARK COMPLETE")
print("=" * 70)

return {
    "p50_ms": round(np.percentile(latencies, 50), 3),
```

```
"p95_ms": round(np.percentile(latencies, 95), 3),  
"p99_ms": round(np.percentile(latencies, 99), 3),  
"max_throughput": throughput_results[-1]["throughput_per_sec"],  
}
```

```
if __name__ == "__main__":  
    run_benchmark()
```

FILE: src/cost_calculator.py

```
"""Hardware cost comparison calculator for ML inference deployment."""  
from tabulate import tabulate  
# Hardware pricing database (approximate cloud pricing, 2024)  
HARDWARE_OPTIONS = {  
    "CPU (2 vCPU)": {  
        "cost_per_hour": 0.05,  
        "max_throughput": 200, # inferences/sec (small model)  
        "typical_latency_ms": 5,  
        "memory_gb": 4,  
        "type": "cpu",  
    },  
    "CPU (8 vCPU)": {  
        "cost_per_hour": 0.17,  
        "max_throughput": 800,  
        "typical_latency_ms": 3,  
        "memory_gb": 16,  
        "type": "cpu",  
    },  
    "CPU (16 vCPU)": {  
        "cost_per_hour": 0.34,  
        "max_throughput": 1500,  
        "typical_latency_ms": 2,  
        "memory_gb": 32,  
        "type": "cpu",  
    },  
    "GPU (T4)": {  
        "cost_per_hour": 1.50,  
        "max_throughput": 5000,  
        "typical_latency_ms": 2,  
        "memory_gb": 16,  
        "type": "gpu",  
    },  
}
```

```
"GPU (A10G)": {
    "cost_per_hour": 3.50,
    "max_throughput": 10000,
    "typical_latency_ms": 1,
    "memory_gb": 24,
    "type": "gpu",
},
"GPU (A100)": {
    "cost_per_hour": 12.00,
    "max_throughput": 30000,
    "typical_latency_ms": 0.5,
    "memory_gb": 80,
    "type": "gpu",
},
}

# Pricing modifiers
SPOT_DISCOUNT = 0.30    # Spot = 30% of on-demand
RESERVED_1YR = 0.60      # 1-year reserved = 60% of on-demand
RESERVED_3YR = 0.40      # 3-year reserved = 40% of on-demand

def calculate_costs(
    required_throughput: int,
    max_latency_ms: float,
    model_memory_gb: float = 1.0,
    requests_per_day: int = 1_000_000,
    hours_per_day: int = 24,
):
    """Calculate monthly costs across all hardware options."""
    results = []
    monthly_requests = requests_per_day * 30

    for name, specs in HARDWARE_OPTIONS.items():
        # Check if hardware meets requirements
        meets_throughput = specs["max_throughput"] >= required_throughput
        meets_latency = specs["typical_latency_ms"] <= max_latency_ms
        meets_memory = specs["memory_gb"] >= model_memory_gb

        # Calculate instances needed (if throughput exceeds single instance)
        instances_needed = max(1, required_throughput // specs["max_throughput"] + 1)
        if meets_throughput:
            instances_needed = 1
```

```
# Monthly costs
monthly_hours = hours_per_day * 30
on_demand_monthly = specs["cost_per_hour"] * monthly_hours * instances_needed
spot_monthly = on_demand_monthly * SPOT_DISCOUNT
reserved_1yr_monthly = on_demand_monthly * RESERVED_1YR
reserved_3yr_monthly = on_demand_monthly * RESERVED_3YR

# Cost per 1000 inferences
cost_per_1k = (on_demand_monthly / monthly_requests) * 1000 if monthly_requests > 0 else 0

viable = meets_throughput and meets_latency and meets_memory
results.append({
    "hardware": name,
    "viable": viable,
    "on_demand": round(on_demand_monthly, 2),
    "spot": round(spot_monthly, 2),
    "reserved_1yr": round(reserved_1yr_monthly, 2),
    "reserved_3yr": round(reserved_3yr_monthly, 2),
    "cost_per_1k": round(cost_per_1k, 5),
    "meets_throughput": "✓" if meets_throughput else "✗",
    "meets_latency": "✓" if meets_latency else "✗",
    "meets_memory": "✓" if meets_memory else "✗",
    "instances": instances_needed,
})

return results

def display_results(results, required_throughput, max_latency_ms, requests_per_day):
    """Display cost comparison table."""
    print("=" * 90)
    print("ML DEPLOYMENT COST COMPARISON")
    print("=" * 90)
    print(f"\nRequirements:")
    print(f" Throughput: {required_throughput} inferences/sec")
    print(f" Max Latency: {max_latency_ms} ms")
    print(f" Daily Requests: {requests_per_day:,}")
    print(f" Monthly Requests: {requests_per_day * 30:,}")

# Viable options
print(f"\n{'-' * 90}")
```



```
print("VIABLE OPTIONS (meets all requirements)")
print("-" * 90)

viable = [r for r in results if r["viable"]]
if not viable:
    print(" No hardware option meets all requirements! Consider:")
    print(" - Reducing throughput requirement")
    print(" - Increasing latency tolerance")
    print(" - Using model optimization (quantization)")
else:
    table_data = [
        [r["hardware"], f"${r['on_demand']:.2f}", f"${r['spot']:.2f}",
         f"${r['reserved_1yr']:.2f}", f"${r['reserved_3yr']:.2f}",
         f"${r['cost_per_1k']:.5f}"]
        for r in viable
    ]
    print(tabulate(
        table_data,
        headers=["Hardware", "On-Demand/mo", "Spot/mo", "Reserved 1yr/mo",
                 "Reserved 3yr/mo", "Cost/1K inferences"],
        tablefmt="grid",
    ))

# Recommendation
if viable:
    cheapest = min(viable, key=lambda x: x["on_demand"])
    print(f"\n{'-' * 90}")
    print(f"RECOMMENDATION: {cheapest['hardware']}")
    print(f" On-demand: ${cheapest['on_demand']:.2f}/month")
    print(f" With 3yr reserved: ${cheapest['reserved_3yr']:.2f}/month "
          f"(save ${cheapest['on_demand'] - cheapest['reserved_3yr']:.2f}/month)")
    print(f" Cost per 1K inferences: ${cheapest['cost_per_1k']:.5f}")

# All options comparison
print(f"\n{'-' * 90}")
print("ALL OPTIONS (including non-viable)")
print("-" * 90)
table_data = [
    [r["hardware"], r["meets_throughput"], r["meets_latency"],
     r["meets_memory"], f"${r['on_demand']:.2f}",
     "✓ VIABLE" if r["viable"] else "✗"]
    for r in results
]
```

```
]
print(tabulate(
    table_data,
    headers=["Hardware", "Throughput", "Latency", "Memory", "On-Demand/mo", "Viable"],
    tablefmt="grid",
))

print(f"\n{'=' * 90}")

def run_cost_analysis():
    """Run cost analysis with default parameters."""
    # Scenario: Medium model, moderate throughput
    required_throughput = 500    # 500 inferences/second
    max_latency_ms = 10.0       # 10ms max
    model_memory_gb = 2.0       # 2GB model
    requests_per_day = 1_000_000 # 1M requests/day

    results = calculate_costs(
        required_throughput=required_throughput,
        max_latency_ms=max_latency_ms,
        model_memory_gb=model_memory_gb,
        requests_per_day=requests_per_day,
    )

    display_results(results, required_throughput, max_latency_ms, requests_per_day)
    return results

if __name__ == "__main__":
    run_cost_analysis()
```

FILE: src/tco_analysis.py

```
"""Total Cost of Ownership (TCO) analysis for ML deployments."""
from tabulate import tabulate
def calculate_tco(
    # Traffic parameters
    requests_per_day: int = 1_000_000,
    # Compute parameters
    compute_cost_per_hour: float = 1.50, # e.g., T4 GPU
    instances: int = 1,
    hours_per_day: int = 24,
    # Training costs (periodic retraining)
    training_hours_per_month: float = 10,
```



```
training_cost_per_hour: float = 12.0, # A100 for training
# Storage
model_storage_gb: float = 5,
data_storage_gb: float = 100,
log_storage_gb: float = 50,
storage_cost_per_gb: float = 0.023, # S3 standard
# Networking
data_transfer_gb_per_month: float = 500,
transfer_cost_per_gb: float = 0.09,
load_balancer_monthly: float = 25,
# People
ml_engineer_hours_per_month: float = 40,
devops_hours_per_month: float = 20,
engineer_hourly_rate: float = 80,
# Tools
monitoring_monthly: float = 50,
registry_monthly: float = 20,
ci_cd_monthly: float = 0, # GitHub Actions free tier
# Overhead factor
overhead_factor: float = 1.2,
):
    """Calculate complete TCO breakdown."""

    # Compute costs
    inference_monthly = compute_cost_per_hour * hours_per_day * 30 * instances
    training_monthly = training_hours_per_month * training_cost_per_hour
    compute_total = inference_monthly + training_monthly

    # Storage costs
    total_storage_gb = model_storage_gb + data_storage_gb + log_storage_gb
    storage_total = total_storage_gb * storage_cost_per_gb

    # Networking costs
    transfer_cost = data_transfer_gb_per_month * transfer_cost_per_gb
    network_total = transfer_cost + load_balancer_monthly

    # People costs
    ml_eng_cost = ml_engineer_hours_per_month * engineer_hourly_rate
    devops_cost = devops_hours_per_month * engineer_hourly_rate
    people_total = ml_eng_cost + devops_cost

    # Tools costs
```

```
tools_total = monitoring_monthly + registry_monthly + ci_cd_monthly
```

```
# Subtotal
```

```
subtotal = compute_total + storage_total + network_total + people_total + tools_total
```

```
# Apply overhead
```

```
total_monthly = subtotal * overhead_factor
```

```
# Derived metrics
```

```
monthly_requests = requests_per_day * 30
```

```
cost_per_inference = total_monthly / monthly_requests if monthly_requests > 0 else 0
```

```
cost_per_1k = cost_per_inference * 1000
```

```
return {
```

```
    "compute": {
```

```
        "inference": round(inference_monthly, 2),
```

```
        "training": round(training_monthly, 2),
```

```
        "total": round(compute_total, 2),
```

```
    },
```

```
    "storage": {
```

```
        "total_gb": total_storage_gb,
```

```
        "total": round(storage_total, 2),
```

```
    },
```

```
    "networking": {
```

```
        "transfer": round(transfer_cost, 2),
```

```
        "load_balancer": load_balancer_monthly,
```

```
        "total": round(network_total, 2),
```

```
    },
```

```
    "people": {
```

```
        "ml_engineer": round(ml_eng_cost, 2),
```

```
        "devops": round(devops_cost, 2),
```

```
        "total": round(people_total, 2),
```

```
    },
```

```
    "tools": {
```

```
        "total": round(tools_total, 2),
```

```
    },
```

```
    "subtotal": round(subtotal, 2),
```

```
    "overhead_factor": overhead_factor,
```

```
    "total_monthly": round(total_monthly, 2),
```

```
    "total_annual": round(total_monthly * 12, 2),
```

```
    "total_3yr": round(total_monthly * 36, 2),
```

```
    "cost_per_inference": round(cost_per_inference, 8),
```

```
"cost_per_1k": round(cost_per_1k, 5),  
"monthly_requests": monthly_requests,  
}
```

```
def compare_scenarios():
```

```
    """Compare Cloud vs Self-Hosted vs Hybrid TCO."""
```

```
    print("=" * 80)
```

```
    print("TOTAL COST OF OWNERSHIP (TCO) ANALYSIS")
```

```
    print("=" * 80)
```

```
    print(f"\nScenario: ML Model serving 1,000,000 requests/day")
```

```
    print(f"Model: Medium complexity (Random Forest / Small Neural Net)")
```

```
    print(f"Team: 1 ML Engineer (part-time) + DevOps support\n")
```

```
    # Scenario A: Cloud (GPU)
```

```
    cloud_gpu = calculate_tco(  
        requests_per_day=1_000_000,  
        compute_cost_per_hour=1.50,    # T4  
        instances=1,  
        ml_engineer_hours_per_month=40,  
        devops_hours_per_month=10,    # Less DevOps for managed cloud  
    )
```

```
    # Scenario B: Cloud (CPU - optimized model)
```

```
    cloud_cpu = calculate_tco(  
        requests_per_day=1_000_000,  
        compute_cost_per_hour=0.17,    # 8 vCPU  
        instances=2,                    # Need 2 for throughput  
        ml_engineer_hours_per_month=40,  
        devops_hours_per_month=10,  
    )
```

```
    # Scenario C: Self-hosted GPU
```

```
    # Amortize $5000 hardware over 36 months = ~$139/month
```

```
    self_hosted = calculate_tco(  
        requests_per_day=1_000_000,  
        compute_cost_per_hour=0.19,    # Amortized hardware + power: ~$139/mo / 720hr  
        instances=1,  
        ml_engineer_hours_per_month=40,  
        devops_hours_per_month=40,    # More DevOps for self-hosted!  
        monitoring_monthly=0,          # Self-hosted Prometheus (free)
```

```
load_balancer_monthly=0,    # Self-managed
)

# Display comparison
scenarios = {
    "Cloud GPU (T4)": cloud_gpu,
    "Cloud CPU (optimized)": cloud_cpu,
    "Self-Hosted GPU": self_hosted,
}

# Summary table
print("-" * 80)
print("MONTHLY COST COMPARISON")
print("-" * 80)

table_data = []
for name, data in scenarios.items():
    table_data.append([
        name,
        f"${data['compute']['total']:.2f}",
        f"${data['people']['total']:.2f}",
        f"${data['storage']['total']:.2f}",
        f"${data['networking']['total']:.2f}",
        f"${data['tools']['total']:.2f}",
        f"${data['total_monthly']:.2f}",
    ])

print(tabulate(
    table_data,
    headers=["Scenario", "Compute", "People", "Storage", "Network",
            "Tools", "TOTAL/month"],
    tablefmt="grid",
))

# Per-inference cost
print(f"\n{'-' * 80}")
print("COST PER 1,000 INFERENCE")
print("-" * 80)

for name, data in scenarios.items():
    print(f" {name:25s}: ${data['cost_per_1k']:.5f}")
```

```
# Annual comparison
print(f"\n{'-' * 80}")
print("MULTI-YEAR PROJECTION")
print("-" * 80)

table_data = [
    [name, f"${data['total_monthly']:.2f}",
     f"${data['total_annual']:.2f}", f"${data['total_3yr']:.2f}"]
    for name, data in scenarios.items()
]

print(tabulate(
    table_data,
    headers=["Scenario", "Monthly", "Annual", "3-Year"],
    tablefmt="grid",
))

# Detailed breakdown of recommended option
cheapest_name = min(scenarios, key=lambda k: scenarios[k]["total_monthly"])
cheapest = scenarios[cheapest_name]

print(f"\n{'-' * 80}")
print(f"DETAILED BREAKDOWN — {cheapest_name} (Lowest TCO)")
print("-" * 80)

breakdown = [
    ["Compute (inference)", f"${cheapest['compute']['inference']:.2f}",
     f"${cheapest['compute']['inference']/cheapest['subtotal']*100:.0f}%"],
    ["Compute (training)", f"${cheapest['compute']['training']:.2f}",
     f"${cheapest['compute']['training']/cheapest['subtotal']*100:.0f}%"],
    ["People (ML Engineer)", f"${cheapest['people']['ml_engineer']:.2f}",
     f"${cheapest['people']['ml_engineer']/cheapest['subtotal']*100:.0f}%"],
    ["People (DevOps)", f"${cheapest['people']['devops']:.2f}",
     f"${cheapest['people']['devops']/cheapest['subtotal']*100:.0f}%"],
    ["Storage", f"${cheapest['storage']['total']:.2f}",
     f"${cheapest['storage']['total']/cheapest['subtotal']*100:.0f}%"],
    ["Networking", f"${cheapest['networking']['total']:.2f}",
     f"${cheapest['networking']['total']/cheapest['subtotal']*100:.0f}%"],
    ["Tools", f"${cheapest['tools']['total']:.2f}",
     f"${cheapest['tools']['total']/cheapest['subtotal']*100:.0f}%"],
    ["_" * 20, "_" * 10, "_" * 5],
    ["Subtotal", f"${cheapest['subtotal']:.2f}", "100%"],
    ["Overhead (×1.2)", f"${cheapest['total_monthly'] - cheapest['subtotal']:.2f}", "20%"],
```

```
["TOTAL", f"${cheapest['total_monthly']:.2f}", ""],
]
print(tabulate(breakdown, headers=["Component", "Monthly Cost", "% of Base"],
              tablefmt="grid"))

# Key insight
print(f"\n{'=' * 80}")
print("KEY INSIGHTS")
print('=' * 80)
print(f" • People cost: ${cheapest['people']['total']:.2f}/month "
      f"({cheapest['people']['total']/cheapest['subtotal']*100:.0f}% of base cost)")
print(f" • Compute cost: ${cheapest['compute']['total']:.2f}/month "
      f"({cheapest['compute']['total']/cheapest['subtotal']*100:.0f}% of base cost)")
print(f" • People cost {'exceeds' if cheapest['people']['total'] > cheapest['compute']['total'] else 'is less than'} "
      f"compute cost")
print(f" • Automation investment reduces the largest cost component (people)")
print(f" • Cloud CPU with optimized model often beats GPU for small/medium models")
print(f"\n{'=' * 80}")

if __name__ == "__main__":
    compare_scenarios()
```

FILE: run_all.py

```
"""Run all cost analysis tools in sequence."""

import os
import sys

# Ensure we can import from src/
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

def main():
    print("\n" + "=" * 80)
    print(" ML DEPLOYMENT COST ANALYSIS TOOLKIT")
    print("=" * 80 + "\n")

    # Step 1: Train sample model (if not exists)
    model_path = "models/sample_model.pkl"
    if not os.path.exists(model_path):
```




```
print("Step 0: Training sample model for benchmarking...")
print("-" * 80)
from models.train_sample_model import create_sample_model
create_sample_model(model_path)
print()

# Step 2: Run benchmark
print("\nStep 1: INFERENCE BENCHMARK")
print("=" * 80)
from src.benchmark import run_benchmark
benchmark_results = run_benchmark()

# Step 3: Run cost calculator
print("\n\nStep 2: COST COMPARISON")
print("=" * 80)
from src.cost_calculator import run_cost_analysis
run_cost_analysis()

# Step 4: Run TCO analysis
print("\n\nStep 3: TCO ANALYSIS")
print("=" * 80)
from src.tco_analysis import compare_scenarios
compare_scenarios()

# Final summary
print("\n" + "-" * 80)
print(" ANALYSIS COMPLETE")
print("-" * 80)
print(f"\n Benchmark P99 latency: {benchmark_results['p99_ms']:.3f} ms")
print(f" Max throughput measured: {benchmark_results['max_throughput']:.0f} inferences/sec")
print(f"\n These measurements feed into cost calculations above.")
print(f" Change parameters in each script to model YOUR deployment scenario.")
print()

if __name__ == "__main__":
    main()
```

SECTION 5: STEP-BY-STEP SETUP

STEP 1: Create project directory



```
mkdir -p ml-cost-toolkit/src ml-cost-toolkit/models ml-cost-toolkit/data  
cd ml-cost-toolkit
```

STEP 2: Create all files as shown in Section 4

STEP 3: Install dependencies

```
pip install -r requirements.txt
```

STEP 4: Train the sample model

```
python models/train_sample_model.py
```

STEP 5: Run individual tools

```
# Benchmark only
```

```
python src/benchmark.py
```

```
# Cost calculator only
```

```
python src/cost_calculator.py
```

```
# TCO analysis only
```

```
python src/tco_analysis.py
```

STEP 6: Run complete analysis

```
python run_all.py
```

SECTION 6: VERIFICATION STEPS

6A: Verify Model Training

```
python models/train_sample_model.py
```

Expected output:

- Model saved to models/sample_model.pkl
- Features: 20
- Classes: 3
- Estimators: 100
- Model size: ~150-200 KB

6B: Verify Benchmark

python src/benchmark.py

Expected: Table showing P50, P95, P99 latency (typically <1ms for this model on CPU), plus throughput table showing increasing throughput with batch size.

6C: Verify Cost Calculator

python src/cost_calculator.py

Expected: Table comparing 6 hardware options, marking which meet requirements, showing monthly costs across pricing tiers.

6D: Verify TCO Analysis

python src/tco_analysis.py

Expected: 3-scenario comparison (Cloud GPU, Cloud CPU, Self-Hosted), monthly and annual totals, detailed breakdown showing people cost vs compute cost.

6E: Verify Complete Run

python run_all.py

Expected: All three tools run in sequence with clear section headers.
Final summary shows benchmark results and instructions to customise.

SECTION 7: CUSTOMISATION — CHANGING PARAMETERS

To demonstrate how recommendations change with different inputs:

Change 1 — Double traffic:

In cost_calculator.py, change requests_per_day = 2_000_000

Observe: Smaller hardware may no longer meet throughput requirements

Change 2 — Tighter latency:

Change max_latency_ms = 2.0

Observe: CPU options eliminated, only GPU remains viable

Change 3 — Bigger team:

In tco_analysis.py, change ml_engineer_hours_per_month = 80

Observe: People cost dominates even more, strengthening automation argument



Change 4 — Spot pricing:

Note the spot column in cost calculator

For training (interruptible): spot saves 70%

For serving (must be always-on): reserved saves 40-60%

SECTION 8: TROUBLESHOOTING

ISSUE: "ModuleNotFoundError: No module named 'tabulate'"

FIX: pip install tabulate

ISSUE: "FileNotFoundError: models/sample_model.pkl"

FIX: Run "python models/train_sample_model.py" first

ISSUE: "ModuleNotFoundError: No module named 'src'"

FIX: Run scripts from the project root directory (ml-cost-toolkit/)

Or run "python run_all.py" which handles imports

ISSUE: Benchmark numbers seem too fast (< 0.01ms)

NOTE: This is expected! A small scikit-learn model on CPU is very fast.

GPU models (PyTorch/TensorFlow) would show higher latencies.

The benchmark STRUCTURE is what matters for demonstration.

ISSUE: "Permission denied" on Windows

FIX: Run terminal as administrator or use a virtual environment